

```
# Stack implementation using list
```

```
stack = []
```

```
# Push operation
```

```
stack.append(10)
```

```
stack.append(20)
```

```
stack.append(30)
```

```
print("Stack after push:", stack)
```

```
# Pop operation
```

```
removed = stack.pop()
```

```
print("Popped element:", removed)
```

```
print("Stack after pop:", stack)
```

```
# Peek (top element)
```

```
print("Top element:", stack[-1])
```

```
output:
```

```
Stack after push: [10, 20, 30]
```

```
Popped element: 30
```

```
Stack after pop: [10, 20]
```

```
Top element: 20
```

```
# Queue implementation using list
```

```
queue = []
```

```
# Enqueue operation
```

```
queue.append(10)
```

```
queue.append(20)
```

```
queue.append(30)
```

```
print("Queue after enqueue:", queue)
```

```
# Dequeue operation
```

```
removed = queue.pop(0)
```

```
print("Dequeued element:", removed)
```

```
print("Queue after dequeue:", queue)
```

```
# Front element
```

```
print("Front element:", queue[0])
```

Method 2: Using deque (Recommended)

```
from collections import deque

queue = deque()

# Enqueue
queue.append(10)
queue.append(20)
queue.append(30)

print("Queue:", queue)

# Dequeue
removed = queue.popleft()
print("Dequeued element:", removed)

print("Queue after dequeue:", queue)
```

Ordered List Implementation

An Ordered List is a list where elements are always kept sorted.

Concept

- No need to sort later
- Elements are inserted in correct position directly

```
class OrderedList:
    def __init__(self):
        self.items = []

    def insert(self, item):
        # insert in sorted order
        i = 0
        while i < len(self.items) and self.items[i] < item:
            i += 1
        self.items.insert(i, item)
```

```
def display(self):
    print("Ordered List:", self.items)
```

```
# Example usage
ol = OrderedList()
```

```
ol.insert(30)
ol.insert(10)
ol.insert(20)
ol.insert(5)
```

```
ol.display()
```

output:
Ordered List: [5, 10, 20, 30]

Tree Implementation (Binary Tree)

We'll create a simple Binary Tree where each node has:

- left child
- right child

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
# Create nodes
root = Node(10)
root.left = Node(5)
root.right = Node(20)
root.left.left = Node(3)
root.left.right = Node(7)
```

```
# Traversal (Inorder)
def inorder(node):
    if node:
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)
```

```
print("Inorder Traversal:")  
inorder(root)
```

output:

Inorder Traversal:

3 5 7 10 20